ktunotes.in

GET IT ON Google Play



KTU NOTES

The learning companion.

KTU STUDY MATERIALS | SYLLABUS | LIVE NOTIFICATIONS | SOLVED QUESTION PAPERS

Website: www.ktunotes.in

CST414
DEEP LEARNING
Module-4 PART -I

Ktunotes.in

SYLLABUS

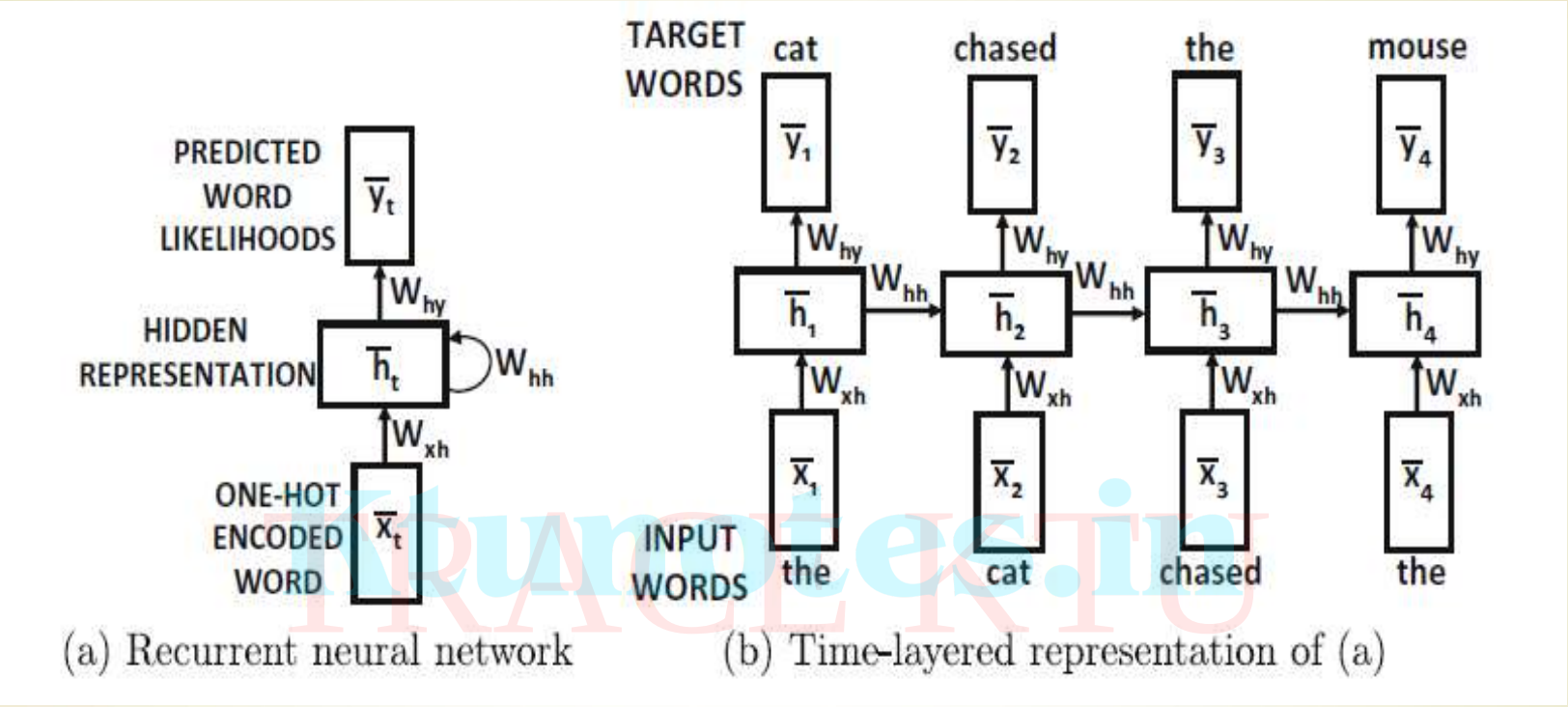
- **Module- 4 (Recurrent Neural Network)**
- **Recurrent neural networks – Computational graphs, RNN design, encoder – decoder sequence to sequence architectures**, deep recurrent networks, recursive neural networks, modern RNNs LSTM and GRU.

Krnotes.in

Recurrent neural networks

- Recurrent neural networks are designed for sequential data like **text sentences, time-series,and other discrete sequences like biological sequences**
- The input is of the form *x*1 . . . *xn*, where *xt* is a *d*-dimensional point received at the time-stamp *t*.
- In a text-setting, the vector xt will contain the **one-hot encoded** word at the t th time-stamp.
- In one-hot encoding, we have a vector of length equal to the lexicon size, and **the component for the relevant word has a value of 1. All other components are 0**
- **successive words are dependent on one another.**

TRACE RTU



- key point is that there is an input *xt* at each time-stamp, and a hidden state *ht* that changes at each time stamp as new data points arrive. Each time-stamp also has an output value *yt*.
- When used in the text-setting of predicting the next word, this approach is referred to as ***language modeling***.

- The hidden state at time t is given by a function of the input vector at time t and the hidden vector at time $(t - 1)$:

$$\overline{h}_t = f(\overline{h}_{t-1}, \overline{x}_t)$$

- A separate function $yt = g(ht)$ is used to learn the output probabilities from the hidden States
- Note that the functions $f(\cdot)$ and $g(\cdot)$ are the same at each time stamp.
- A key point here is the presence of the self-loop in Figure 1.17(a), which will cause the hidden state of the neural network to change after the input of each xt.
- works with sequences of finite length, and it makes sense to unfurl the loop into a "time-layered" network that looks more like a feed-forward network. This network is shown in Figure 1.17(b).

KTUQBANK.in

- weight matrices of the **connections *are shared by multiple connections* in the time layered network to** ensure that the same function is used at each time stamp. This sharing is the key to the domain-specific insights that are learned by the network.
- The backpropagation algorithm takes the sharing and temporal length into account when updating the weights during the learning process. This special type of backpropagation algorithm is referred to as ***backpropagation through time (BPTT).***
- Because of the recursive nature ,the recurrent network has the *ability to compute a function of variable-length inputs*.
- For example, starting at *h*0, which is typically fixed to some constant

vector, we have $\overline{h}_1 = f(\overline{h}_0, \overline{x}_1)$ and $\overline{h}_2 = f(f(\overline{h}_0, \overline{x}_1), \overline{x}_2)$. Note that $\overline{h}_1$ is a function of only $\overline{x}_1$, whereas $\overline{h}_2$ is a function of both $\overline{x}_1$ and $\overline{x}_2$. Since the output $\overline{y}_t$ is a function of $\overline{h}_t$, these properties are inherited by $\overline{y}_t$ as well. In general, we can write the following:

$$\overline{y}_t = F_t(\overline{x}_1, \overline{x}_2, \ldots \overline{x}_t) \tag{1.37}$$

- Note that the function $Ft(\cdot)$ varies with the value of t. Such an approach is particularly useful for variable-length inputs like text sentences.
- **An interesting theoretical property of recurrent neural networks is that they are *Turing complete* . What this means is that *given enough data and computational resources*, a recurrent neural network can simulate any algorithm.**

Krnotes.in

COMPUTATIONAL GRAPHS

- a recurrent neural network is a neural network that is specialized for processing a sequence of values $x(1), \ldots, x(\tau)$
- A computational graph is a way to formalize the structure of a set of computations,
- such as those involved in mapping inputs and parameters to outputs and loss.
- **unfolding** a recursive or recurrent computation into a computational graph
- Unfolding this graph results in the sharing of parameters across a deep networkStructure

Hnotes.in

- For example, consider the classical form of a dynamical system:

$$s^{(t)} = f(s^{(t-1)}; \theta), \tag{10.1}$$

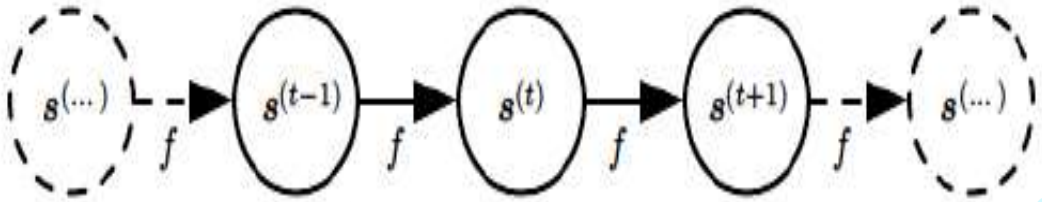
where $s^{(t)}$ is called the state of the system.

- is recurrent because the definition of s at time t refers back to the same definition at time $t - 1$.
- For a finite number of time steps τ , the graph can be unfolded by applying the definition $\tau - 1$ times. For example, if we unfold equation 10.1 for $\tau = 3$ time steps, we obtain.

$$s^{(3)} = f(s^{(2)}; \theta) \tag{10.2}$$

$$= f(f(s^{(1)}; \theta); \theta) \tag{10.3}$$

- Such an expression can now be represented by a traditional **directed acyclic computational graph**



Ktunotes.in

Figure 10.1: The classical dynamical system described by equation 10.1, illustrated as an unfolded computational graph. Each node represents the state at some time t and the function f maps the state at t to the state at $t + 1$. The same parameters (the same value of θ used to parametrize f) are used for all time steps.

- As another example, let us consider a dynamical system driven by an external signal $x(t)$,

$$\boldsymbol{s}^{(t)} = f(\boldsymbol{s}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta}), \qquad (10.4)$$

- any function involving recurrence can be considered a recurrent neural network.
- To indicate that the state is the hidden units of the network, we now rewrite equation 10.4 using the variable h to represent the state:

$$\boldsymbol{h}^{(t)} = f(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta}), \qquad (10.5)$$

TRACE KTU

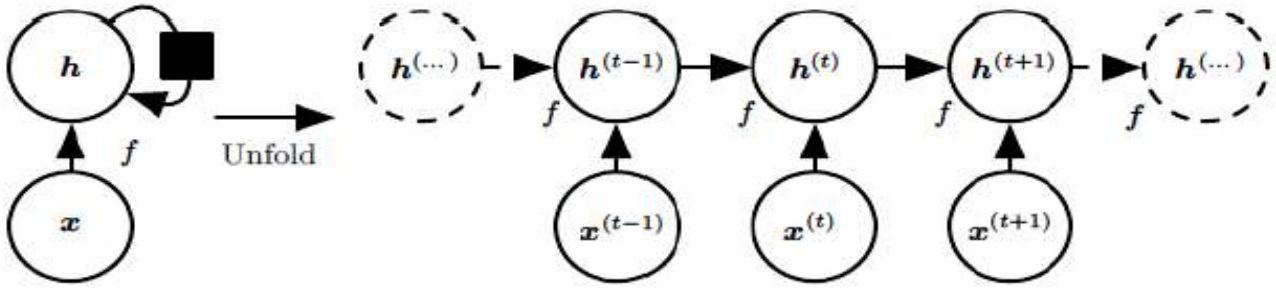


Figure 10.2: A recurrent network with no outputs. This recurrent network just processes information from the input $\boldsymbol{x}$ by incorporating it into the state $\boldsymbol{h}$ that is passed forward through time. *(Left)*Circuit diagram. The black square indicates a delay of a single time step. *(Right)*The same network seen as an unfolded computational graph, where each node is now associated with one particular time instance.

Kractes.in

- One way to draw the RNN is with a diagram containing one node for every component that might exist in a physical implementation of the model, such as a biological neural network
- In this view, the network defines a circuit that operates in real time, with physical parts whose current state can influence their future state,

- The other way to draw the RNN is as an unfolded computational graph, in which each component is represented by many different variables, with one variable per time step, representing the state of the component at that point in time.
- Each variable for each time step is drawn as a separate node of the computational graph, as in the right of figure 10.2.
- The unfolded graph now has a size that depends on the sequence length.

Ktunotes.in

We can represent the unfolded recurrence after t steps with a function $g^{(t)}$:

$$\boldsymbol{h}^{(t)} = g^{(t)}(\boldsymbol{x}^{(t)}, \boldsymbol{x}^{(t-1)}, \boldsymbol{x}^{(t-2)}, \ldots, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(1)}) \quad (10.6)$$

$$= f(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta}) \quad (10.7)$$

- The function ***g*(*t*) takes the whole past sequence (*x*(*t*), *x*(*t*−1), *x*(*t*−2), . . . , *x*(2), *x*(1))** as input and produces the current state,
- unfolded recurrent structure allows us to **factorize *g*(*t*)** into repeated application of a function *f*.
- The **unfolding process thus introduces two major advantages**:
- 1. Regardless of the sequence length, the learned model always has the same input size, because it is specified in terms of **transition from one state to another state**, rather than specified in terms of a variable-length history of states.
- 2. It is possible to use the ***same* transition function *f* with** the same parameters at every time step.

- The recurrent graph and the unrolled graph have their **uses**
- The recurrent graph is succinct.
- The unfolded graph provides an explicit description of which computations to perform.
- The unfolded graph also helps **to illustrate the idea of information flow forward in time** (computing outputs and losses) and backward in time (computing gradients) by explicitly showing the path along which this information flows.

KTUQBANK.IN

RNN Design

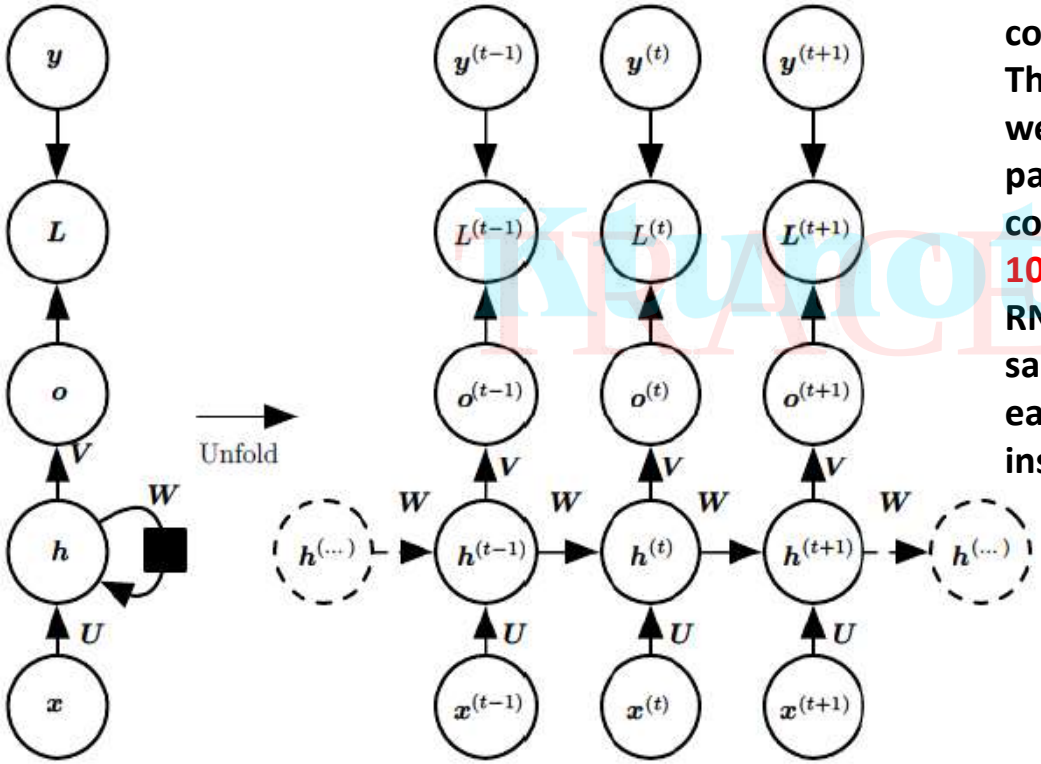


Figure 10.3: The computational graph to compute the training loss of a recurrent network that maps an input sequence of *x* values to a corresponding sequence of output *o* values.
A loss *L* measures how far each *o* is from the corresponding training target *y* . When using softmax outputs, we assume *o* is the unnormalized log probabilities. The loss *L* internally computes ˆ *y* = softmax(*o*) and compares this to the target *y*. The RNN has input to hidden connections parametrized by a weight matrix *U*, hidden-to-hidden recurrent connections parametrized by a weight matrix *W* , and hidden-to-output connections parametrized by a weight matrix *V* . Equation 10.8 defines forward propagation in this model. *(Left)*The RNN and its loss drawn with recurrent connections. *(Right)*The same seen as an time unfolded computational graph, where each node is now associated with one particular time instance.

- Some examples of important design patterns for recurrent neural networks include the following

- • Recurrent networks that produce an output at each time step and have recurrent connections between hidden units, illustrated in figure 10.3.
- • Recurrent networks that produce an output at each time step and have recurrent connections only from the output at one time step to the hidden units at the next time step, illustrated in figure 10.4
- • Recurrent networks with recurrent connections between hidden units, that read an entire sequence and then produce a single output, illustrated in figure 10.5.

KNACE KNOWLEDGE

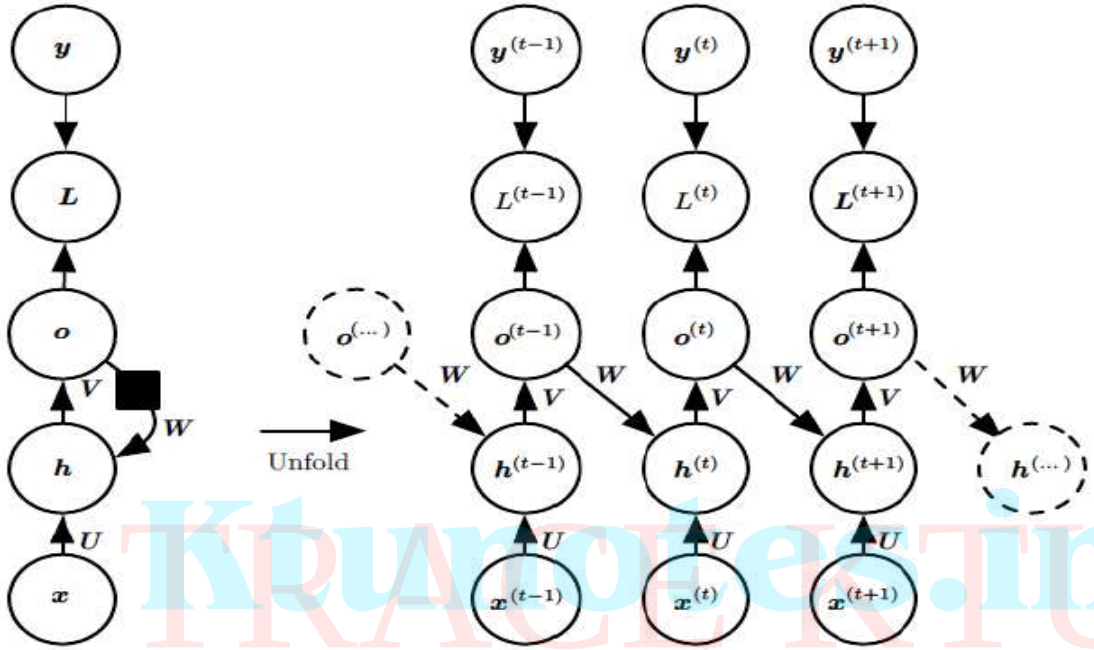


Figure 10.4: An RNN whose only recurrence is the feedback connection from the output to the hidden layer. At each time step t, the input is $\boldsymbol{x}_t$, the hidden layer activations are $\boldsymbol{h}^{(t)}$, the outputs are $\boldsymbol{o}^{(t)}$, the targets are $\boldsymbol{y}^{(t)}$ and the loss is $L^{(t)}$. *(Left)*Circuit diagram. *(Right)*Unfolded computational graph. Such an RNN is less powerful (can express a

smaller set of functions) than those in the family represented by figure 10.3.

- The RNN in this figure is trained to put a specific output value into o , and ***o* is the only information it is allowed to send to the future.**
- There **are no direct connections from *h* going forward**. The previous h is connected to the present only indirectly, via the predictions it was used to produce.
- Unless o is very high-dimensional and rich, it will usually lack important information from the past.
- This makes the RNN in this figure **less powerful**, but it may be easier to train because each time step can be trained in isolation from the others $t = 1$ to $t = \tau$, we apply the following update equations:

$$\boldsymbol{a}^{(t)} = \boldsymbol{b} + \boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{U}\boldsymbol{x}^{(t)} \quad (10.8)$$

$$\boldsymbol{h}^{(t)} = \tanh(\boldsymbol{a}^{(t)}) \quad (10.9)$$

$$\boldsymbol{o}^{(t)} = \boldsymbol{c} + \boldsymbol{V}\boldsymbol{h}^{(t)} \quad (10.10)$$

$$\hat{\boldsymbol{y}}^{(t)} = \text{softmax}(\boldsymbol{o}^{(t)}) \quad (10.11)$$

- where the parameters are the bias vectors b and c along with the weight matrices ***U*** **,** ***V*** **and** ***W*** **, respectively for input-to-hidden, hidden-to-output and hidden-to –hidden connections**
- This is an example of a recurrent network that maps an input sequence to an output sequence of the same length.
- The total loss for a given sequence of x values paired with a sequence of y values would then be just
- The sum of the losses over all the time steps. For example, if $L(t)$ is the negative log-likelihood of y (t) given $x(1)$, . . . , $x(t)$, then

$$L\left(\{\boldsymbol{x}^{(1)},\ldots,\boldsymbol{x}^{(\tau)}\},\{\boldsymbol{y}^{(1)},\ldots,\boldsymbol{y}^{(\tau)}\}\right) \tag{10.12}$$

$$=\sum_t L^{(t)} \tag{10.13}$$

$$=-\sum_t \log p_{\text{model}}\left(y^{(t)} \mid \{\boldsymbol{x}^{(1)},\ldots,\boldsymbol{x}^{(t)}\}\right), \tag{10.14}$$

KRMOLES.in

where $p_{\text{model}}\left(y^{(t)} \mid \{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t)}\}\right)$ is given by reading the entry for $y^{(t)}$ from the model's output vector $\hat{\boldsymbol{y}}^{(t)}$. Computing the gradient of this loss function with respect to the parameters is an expensive operation. The gradient computation involves performing a forward propagation pass moving left to right through our illustration of the unrolled graph in figure 10.3, followed by a backward propagation pass

moving right to left through the graph. The runtime is $O(\tau)$ and cannot be reduced by parallelization because the forward propagation graph is inherently sequential; each time step may only be computed after the previous one.

- **States computed in the forward pass must be stored until they are reused during the backward pass, so the memory cost is also $O(\tau)$. The back-propagation algorithm applied to the unrolled graph with $O(\tau)$ cost is called back-propagation through time or BPTT**

KRACKER.in

- **Teacher forcing** is a procedure that emerges from the maximum likelihood criterion, in which during training the y(t) model receives the ground truth output as input at time t + 1.

The conditional maximum likelihood criterion is

$$\log p\left(\boldsymbol{y}^{(1)}, \boldsymbol{y}^{(2)} \mid \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}\right)$$
$$= \log p\left(\boldsymbol{y}^{(2)} \mid \boldsymbol{y}^{(1)}, \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}\right) + \log p\left(\boldsymbol{y}^{(1)} \mid \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}\right)$$

KRAnotes.in

- we see that at time t = 2, the model is trained to maximize the y(2) conditional probability of given both the x sequence so far and the previous y value from the training set.
- Maximum likelihood thus specifies that during training, rather than feeding the model's own output back into itself, these connections should be fed with the target values specifying what the correct output should be.

- motivated teacher forcing as allowing us to avoid back-propagation through time in models that lack hidden-to-hidden connections
- Some models may thus be trained with both teacher forcing and BPTT.
- The disadvantage of strict teacher forcing arises if the network is going to be later used in an open-loop mode, with the network outputs (or samples from the output distribution) fed back as input

TRACE RTU

ENCODER – DECODER SEQUENCE TO SEQUENCE ARCHITECTURES

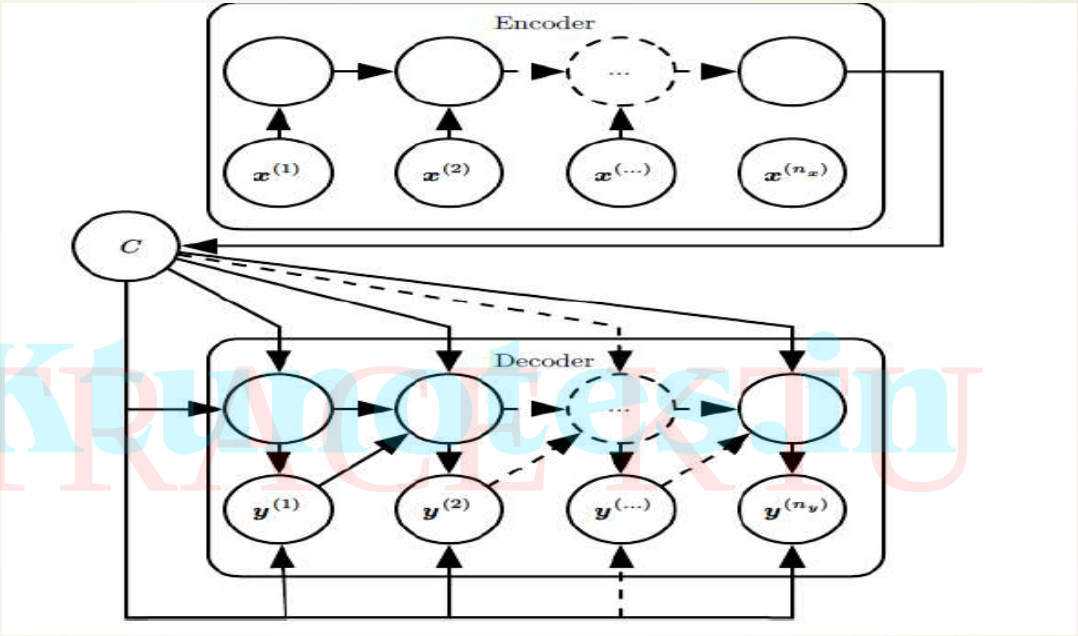


Figure 10.12: Example of an encoder-decoder or sequence-to-sequence RNN architecture, for learning to generate an output sequence $(\mathrm{y}^{(1)}, \ldots, \mathrm{y}^{(n_y)})$ given an input sequence $(\mathrm{x}^{(1)}, \mathrm{x}^{(2)}, \ldots, \mathrm{x}^{(n_x)})$. It is composed of an encoder RNN that reads the input sequence and a decoder RNN that generates the output sequence (or computes the probability of a given output sequence). The final hidden state of the encoder RNN is used to compute a generally fixed-size context variable C which represents a semantic summary of the input sequence and is given as input to the decoder RNN.

- Here we discuss how an RNN can be trained to map an **input sequence to an output sequence which is not necessarily of the same length.**
- This comes up in many applications, such as speech recognition, machine translation or question answering, where the input and output sequences in the training set are generally not of the same length
- The input to the RNN the "context." We want to produce a representation of this context, C . The context C might be a vector or sequence of vectors that summarize the input sequence

$$\boldsymbol{X} = (\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(n_x)}).$$

Ktunotes.in

- (1) an **encoder** or **reader** or **input** RNN processes the input sequence. The encoder emits the context C , usually as a simple function of its final hidden state.
- (2) a **decoder** or **writer** or **output** RNN is conditioned on that fixed-length vector to generate the output sequence

$$\boldsymbol{Y} = (\boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(n_y)}).$$

KRUNOTES.IN

- In a sequence-to-sequence architecture, the two RNNs are trained jointly to maximize the average of

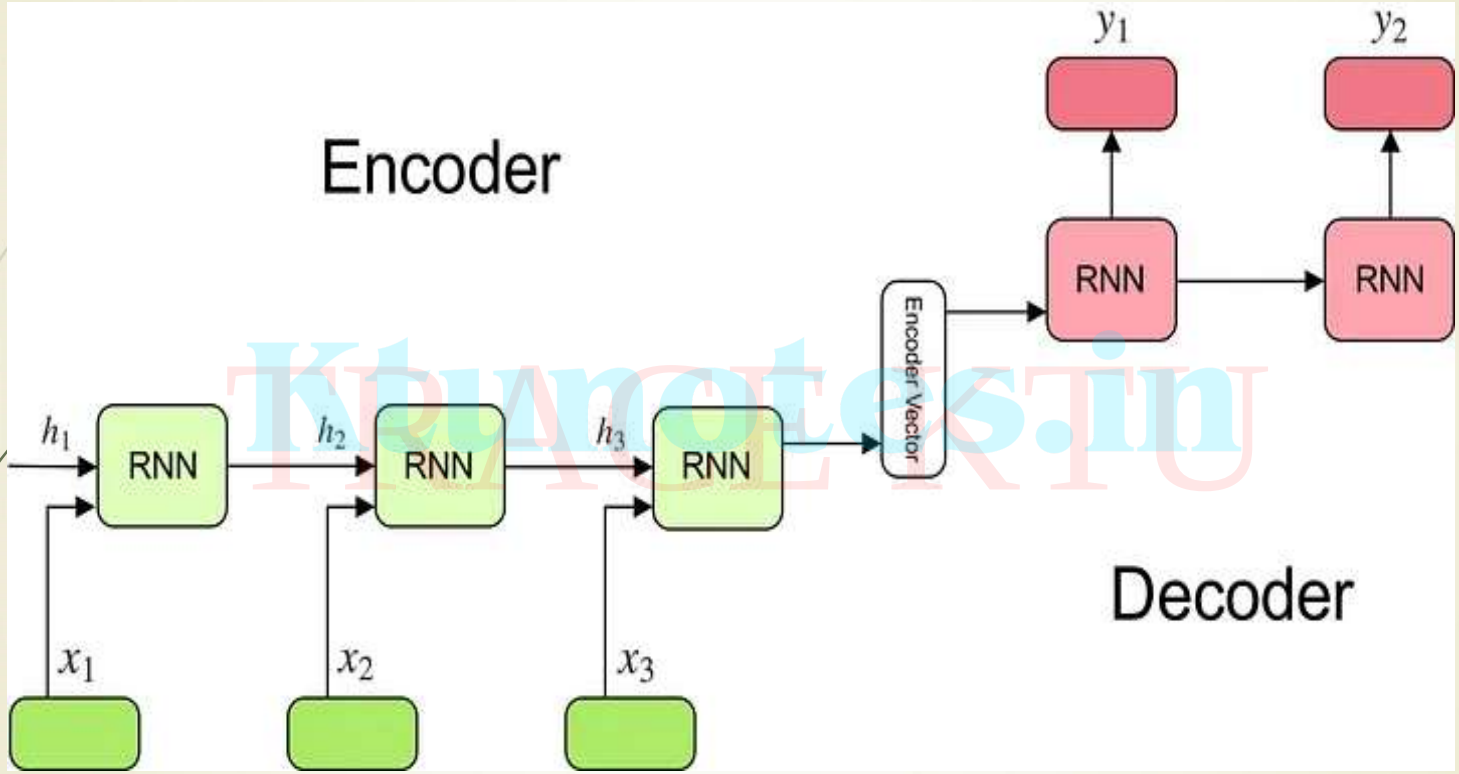
$$\log P(\boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(n_y)} \mid \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(n_x)})$$

over all the pairs of x and y sequences in the training set.

- The **last state *hnx* of the encoder RNN is typically used as a representation *C* of the input sequence** that is provided as input to the decoder RNN.

- If the context *C* is a vector, then the decoder RNN is simply a vector-to sequence RNN
- **There is no constraint that the encoder must have the same size of hidden layer as the decoder**
- One clear limitation of this architecture is when **the context *C* output by the encoder RNN has a dimension that is too small** to properly summarize a long sequence.
- Phenomenon was observed by bahdanau *et al.* (2015) in the context of machine translation. They proposed to make *C* a variable-length sequence rather than a fixed-size vector.
- They introduced an **attention mechanism** that learns to associate elements of the sequence *c* to elements of the output sequence

Ktunotes.in


Encoder
Decoder
RNN
RNN
RNN
RNN
RNN
Encoder Vector
h_1
h_2
h_3
x_1
x_2
x_3
y_1
y_2

Ktunotes.in

- **Encoder**

- A stack of several recurrent units where each accepts a single element of the input sequence, collects information for that element and propagates it forward.
- In question-answering problem, the input sequence is a collection of all words from the question. Each word is represented as x_i where i is the order of that word.
- The hidden states h_i are computed using the formula:

$$h_t = f(W^{(hh)} h_{t-1} + W^{(hx)} x_t)$$

- **Encoder Vector**

- This is the final hidden state produced from the encoder part of the model. It is calculated using the formula above.
- This vector aims to encapsulate the information for all input elements in order to help the decoder make accurate predictions.
- It acts as the initial hidden state of the decoder part of the model.

TRACE RTU

Decoder

- A stack of several recurrent units where each predicts an output *y_t* at a time step *t*.
- Each recurrent unit accepts a hidden state from the previous unit and produces and output as well as its own hidden state.
- In the question-answering problem, the output sequence is a collection of all words from the answer. Each word is represented as *y_i* where *i* is the order of that word.
- Any hidden state *h_i* is computed using the formula:

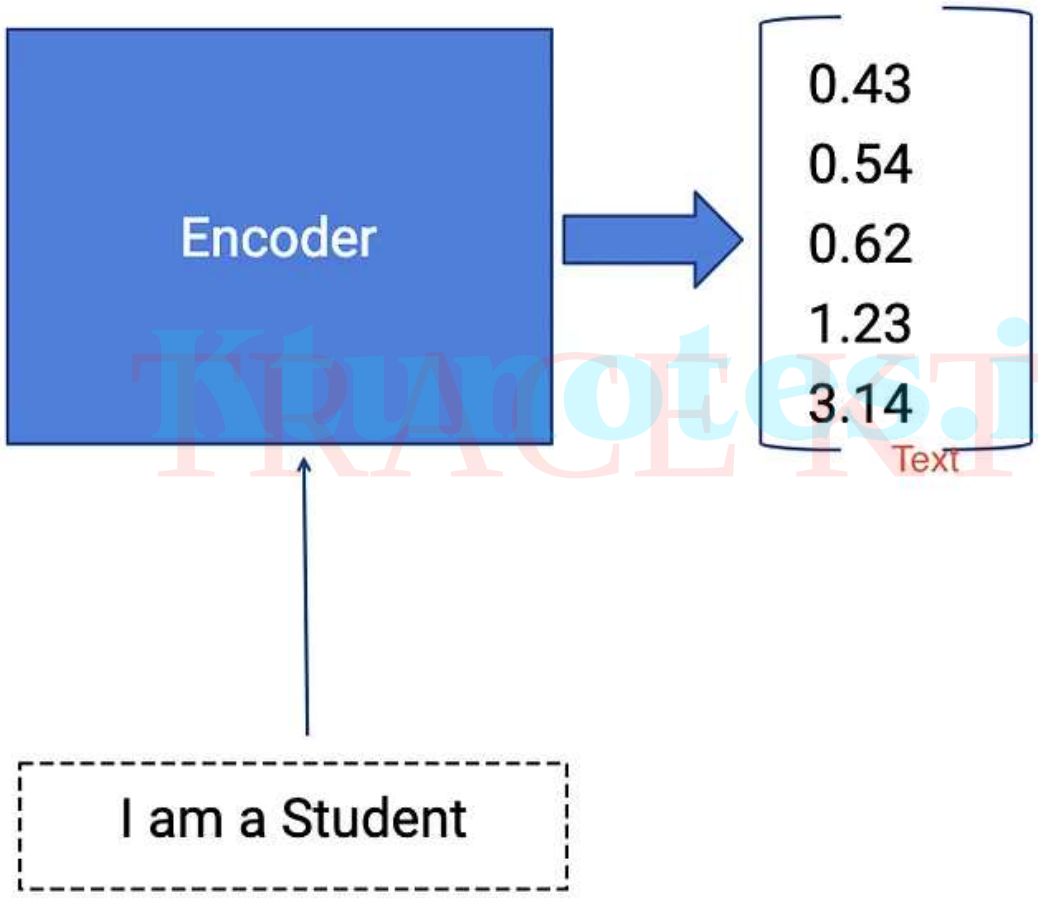
$$h_t = f(W^{(hh)} h_{t-1})$$

- we are just using the previous hidden state to compute the next one.
- The output *y_t* at time step *t* is computed using the formula:

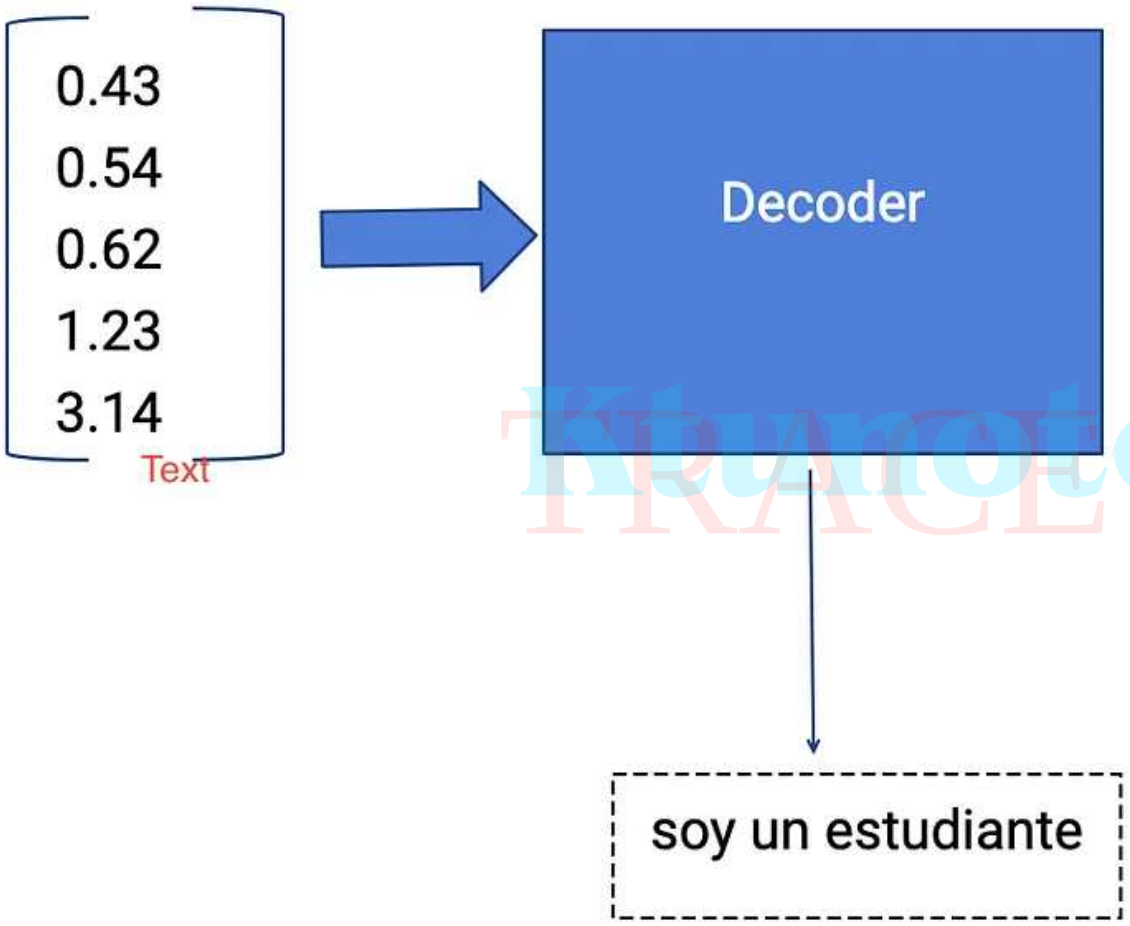
$$y_t = softmax(W^S h_t)$$

- We calculate the outputs using the hidden state at the current time step together with the respective weight W(S). Softmax is used to create a probability vector which will help us determine the final output (e.g. word in the question-answering problem)

Kpuc0-es.ing



Fig; machine translation
English to spanish



Applications

It possesses many applications such as

- Google's Machine Translation
- Question answering chatbots
- Speech recognition
- Time Series Application etc.,

CST414
DEEP LEARNING
Module-4 PART -II

Ktunotes.in

SYLLABUS

- **Module- 4 (Recurrent Neural Network)**
- Recurrent neural networks – Computational graphs, RNN design, encoder – decoder sequence to sequence architectures, **deep recurrent networks, recursive neural networks, modern RNNs LSTM and GRU**.

Ktunotes.in

Deep Recurrent Networks

- The computation in most RNNs can be decomposed into three blocks of parameters and associated transformations:
- 1. from the input to the hidden state,
- 2. from the previous hidden state to the next hidden state, and
- 3. from the hidden state to the output.

Ktunotes.in

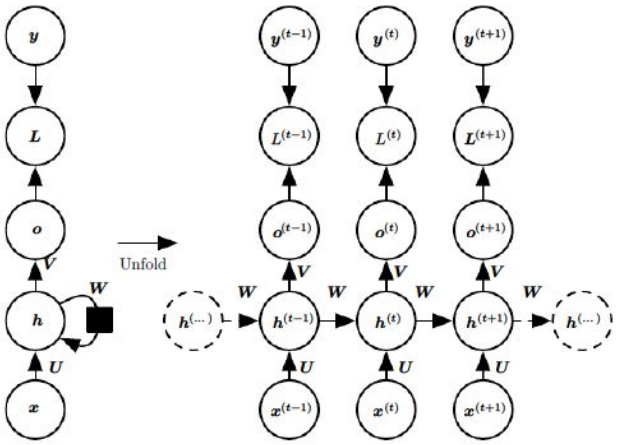


Figure 10.3: The computational graph to compute the training loss of a recurrent network that maps an input sequence of $\boldsymbol{x}$ values to a corresponding sequence of output $\boldsymbol{o}$ values. A loss L measures how far each $\boldsymbol{o}$ is from the corresponding training target $\boldsymbol{y}$. When using softmax outputs, we assume $\boldsymbol{o}$ is the unnormalized log probabilities. The loss L internally computes $\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{o})$ and compares this to the target $\boldsymbol{y}$. The RNN has input to hidden connections parametrized by a weight matrix $\boldsymbol{U}$, hidden-to-hidden recurrent connections parametrized by a weight matrix $\boldsymbol{W}$, and hidden-to-output connections parametrized by a weight matrix $\boldsymbol{V}$. Equation 10.8 defines forward propagation in this model. *(Left)*The RNN and its loss drawn with recurrent connections. *(Right)*The same seen as an time-unfolded computational graph, where each node is now associated with one particular time instance.

Ktunotes.in
TRACE KTU

- When the **network is unfolded**, each of these corresponds to a shallow transformation.
- By a **shallow transformation**, we mean a transformation that would be **represented by a single layer within a deep MLP**.

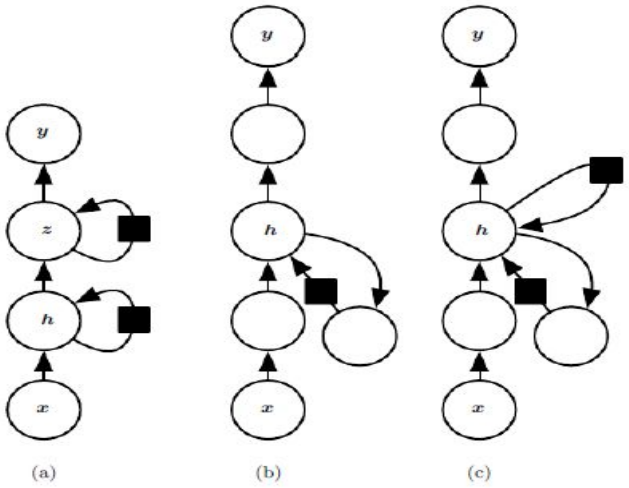


Figure 10.13: A recurrent neural network can be made deep in many ways (Pascanu *et al.*, 2014a). *(a)*The hidden recurrent state can be broken down into groups organized hierarchically. *(b)*Deeper computation (e.g., an MLP) can be introduced in the input-to-hidden, hidden-to-hidden and hidden-to-output parts. This may lengthen the shortest

Ktunotes.in
TRACE KTU

- First to show a significant benefit of decomposing the state of an RNN into multiple layers
- The **lower layers** in the hierarchy depicted in figure ,as playing a **role in transforming the raw input into a representation** that is more appropriate, at the higher levels of the hidden state
- It is easier to optimize shallower architectures, and adding the extra depth of figure 10.13b makes the shortest path from a variable in time step t to a variable in time step $t + 1$ become longer

Ktunotes.in

Recursive Neural Networks

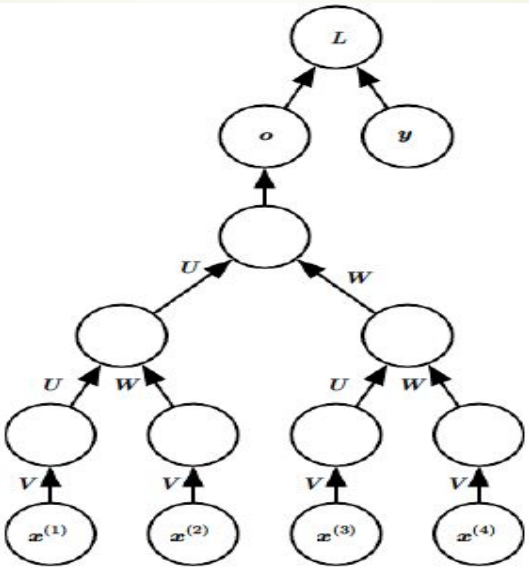


Figure 10.14: A recursive network has a computational graph that generalizes that of the recurrent network from a chain to a tree. A variable-size sequence $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$ can be mapped to a fixed-size representation (the output $\boldsymbol{o}$), with a fixed set of parameters (the weight matrices $\boldsymbol{U}$, $\boldsymbol{V}$, $\boldsymbol{W}$). The figure illustrates a supervised learning case in which some target $\boldsymbol{y}$ is provided which is associated with the whole sequence.

Ktunotes.in
TRACE KTU

- Recursive neural networks represent yet another generalization of recurrent networks, with a different kind of computational graph, which is **structured as a deep tree**,
- Recursive neural networks were introduced by Pollack (1990)
- Recursive networks have been successfully applied to processing data structures as input to neural nets (Frasconi et al., 1997, 1998), in natural language processing (Socher et al., 2011a,c, 2013a) as well as in computer vision (Socher et al., 2011b).
- One clear **advantage of recursive nets over recurrent nets is that for a sequence of the same length τ, the depth (measured as the number of compositions of nonlinear operations) can be drastically reduced from τ to $O(\log \tau)$.**

Ktunotes.in

- In some application domains, external methods can suggest the appropriate tree structure. **For example, when processing natural language sentences,** the tree structure for the recursive network can be fixed to the structure of the parse tree of the sentence provided by a natural language parser

Ktunotes.in

MODERN RNNS LSTM AND GRU

- The most effective sequence models used in practical applications are called gated RNNs.
- These include **the long short-term memory and networks based on the gated recurrent unit**
- gated RNNs are based on the idea of creating paths through time that have derivatives that neither vanish nor explode
- Gated RNNs generalize this to connection weights that may change at each time step

Ktunotes.in

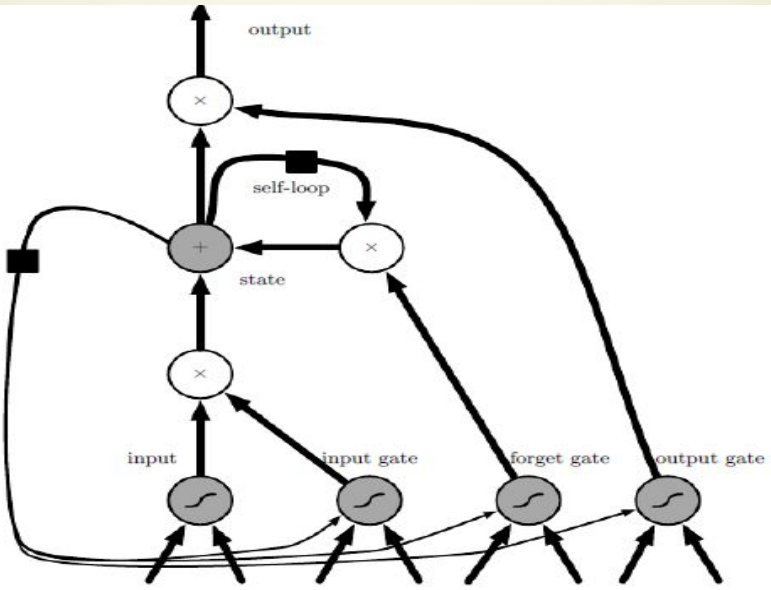
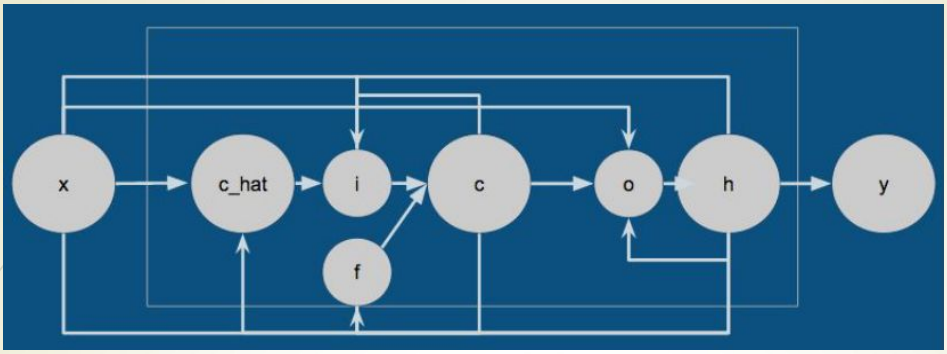


Figure 10.16: Block diagram of the LSTM recurrent network "cell." Cells are connected recurrently to each other, replacing the usual hidden units of ordinary recurrent networks.

Ktunotes.in
TRACE KTU



For the input gate, we have Wxi, Whi, Wci, and bi, which map x(t), h(t-1), c(t-1) to the input gate.

- The 3 gates are called **the input gate, the output gate, and the forget gate.**

$$
\begin{aligned}
i_t &= \sigma(x_t W_{xi} + h_{t-1} W_{hi} + c_{t-1} W_{ci} + b_i) \\
f_t &= \sigma(x_t W_{xf} + h_{t-1} W_{hf} + c_{t-1} W_{cf} + b_f) \\
c_t &= f_t c_{t-1} + i_t \tanh(x_t W_{xc} + h_{t-1} W_{hc} + b_c) \\
o_t &= \sigma(x_t W_{xo} + h_{t-1} W_{ho} + c_t W_{co} + b_o) \\
h_t &= o_t \tanh(c_t)
\end{aligned}
$$

Ktunotes.in

- **The input gate controls how much of the new value goes into the cell, and the forget gate controls how much of the previous cell value goes into the current cell value.**
- For the input gate, we have Wxi, Whi, Wci, and bi, which map x(t), h(t-1) and c(t-1) to the input gate.
- For the forget gate, we have matching parameters, so we have Wxf, Whf, Wcf, and bf, which again map x(t), h(t-1), and c(t-1) to the forget gate.
- Next, to calculate the new cell value, we have Wxc, Whc, and bc, which map x(t) and h(t-1) to the new candidate cell value.
- Finally we have the output gate, which adds 4 more parameters, Wxo, Who, Wco, and bo, which map x(t), h(t-1), and c(t) to the output gate.
- **Notice that the output gate is different from the input and forget gates in that it depends on the cell value at time t, not t-1.**

Ktunotes.in
TRACE KTU

- An input feature is computed with a regular artificial neuron unit. **Its value can be accumulated into the state if the sigmoidal input gate allows** it. The state unit has a linear self-loop whose weight is controlled by the **forget gate**.
- The output of the cell can be shut off by the **output gate**. All the gating units have a sigmoid nonlinearity, while the input unit can have any squashing nonlinearity.
- The **state unit** can also be used as an extra input to the gating units. The black square indicates a delay of a single time step.

LSTM

The clever idea of **introducing self-loops to produce paths where the gradient can flow for long durations is a core contribution** of the initial long short-term memory (LSTM) model

Ktunotes.in

- By making the weight of this self-loop gated (controlled by another hidden unit), the time scale of integration can be changed dynamically
- In many applications, such as unconstrained handwriting recognition, speech recognition, handwriting generation, machine translation, image captioning and parsing .
- Recurrent networks have “LSTM cells” that have an internal recurrence (a self-loop), in addition to the outer recurrence of the RNN.
- **Each cell has the same inputs and outputs as an ordinary recurrent network, but has more parameters and a system of gating units that controls the flow of information**.

Ktunotes.in

- The self-loop weight (or the f(t) associated time constant) is **controlled by a forget gate unit** $f_i^{(t)}$ (for time step t and cell i)), that sets this weight to a value between 0 and 1 via a sigmoid unit:

$$f_i^{(t)} = \sigma\left(b_i^f + \sum_j U_{i,j}^f x_j^{(t)} + \sum_j W_{i,j}^f h_j^{(t-1)}\right), \qquad (10.40)$$

where $\boldsymbol{x}^{(t)}$ is the current input vector and $\boldsymbol{h}^{(t)}$ is the current hidden layer vector, containing the outputs of all the LSTM cells, and $\boldsymbol{b}^f$,$\boldsymbol{U}^f$, $\boldsymbol{W}^f$ are respectively biases, input weights and recurrent weights for the forget gates. The LSTM cell

internal state is thus updated as follows, but with $f_i^{(t)}$ conditional self-loop weight

Ktunotes.in

$$s_i^{(t)} = f_i^{(t)} s_i^{(t-1)} + g_i^{(t)} \sigma \left(b_i + \sum_j U_{i,j} x_j^{(t)} + \sum_j W_{i,j} h_j^{(t-1)} \right), \qquad (10.41)$$

where b, U and W respectively denote the biases, input weights and recurrent weights into the LSTM cell.

The **external input gate** unit $g(t)$ i is computed similarly to the forget gate (with a sigmoid unit to obtain a gating value between 0 and 1), but with its own parameters:

$$g_i^{(t)} = \sigma \left(b_i^g + \sum_j U_{i,j}^g x_j^{(t)} + \sum_j W_{i,j}^g h_j^{(t-1)} \right). \qquad (10.42)$$

Ktunotes.in

- The output $h(t)$ i of the LSTM cell can also be shut off, via the **output gate** q (t) i , which also uses a sigmoid unit for gating:

$$h_i^{(t)} = \tanh\left(s_i^{(t)}\right) q_i^{(t)} \tag{10.43}$$

$$q_i^{(t)} = \sigma\left(b_i^o + \sum_j U_{i,j}^o x_j^{(t)} + \sum_j W_{i,j}^o h_j^{(t-1)}\right) \tag{10.44}$$

  which has parameters bo, $U\,o$, $W\,o$ for its biases, input weights and recurrent weights, respectively

- Among the variants, one can choose to use the cell state $s(t)I$ as an extra input (with its weight) into the three gates of the i-th unit, as shown in figure 10.16

Ktunotes.in

Other Gated RNNs(GATED RECURRENT UNITS OR GRUs)

- The main difference with the LSTM is that a single gating unit simultaneously controls the forgetting factor and the decision to update the state unit
- The update equations are the following:

$$h_i^{(t)} = u_i^{(t-1)} h_i^{(t-1)} + (1 - u_i^{(t-1)}) \sigma \left(b_i + \sum_j U_{i,j} x_j^{(t-1)} + \sum_j W_{i,j} r_j^{(t-1)} h_j^{(t-1)} \right), \tag{10.45}$$

- where *u* stands for "update" gate and *r* for "reset" gate. Their value is defined as usual:

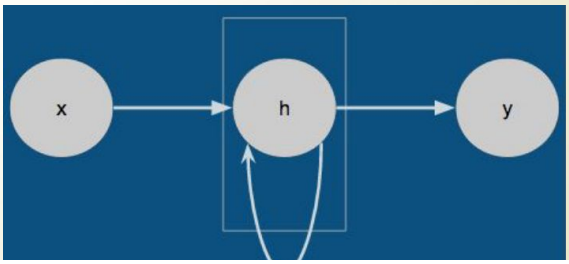
$$u_i^{(t)} = \sigma \left(b_i^u + \sum_j U_{i,j}^u x_j^{(t)} + \sum_j W_{i,j}^u h_j^{(t)} \right) \tag{10.46}$$

$$r_i^{(t)} = \sigma \left(b_i^r + \sum_j U_{i,j}^r x_j^{(t)} + \sum_j W_{i,j}^r h_j^{(t)} \right). \tag{10.47}$$

Ktunotes.in
TRACE KTU

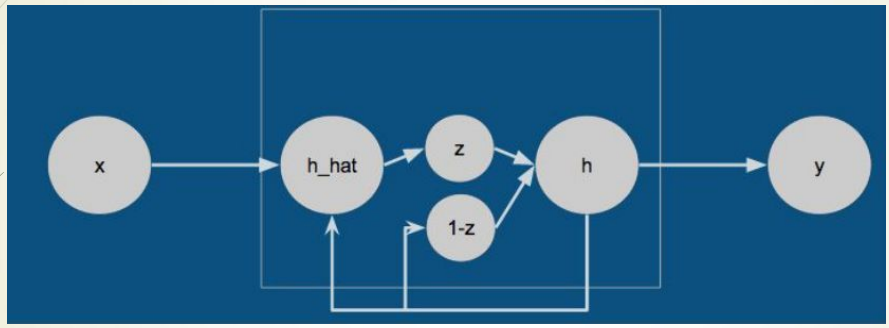
Other Gated RNNs(GATED RECURRENT UNITS OR GRUs)

- The GRU as a simpler version of the LSTM.
- It incorporates a lot of the same concepts, but it has a much smaller number of parameters, and so it can train faster at a constant hidden layer size.
- Recent research has also shown that the accuracy between the LSTM and GRU is comparable and even better with the GRU in some cases.
- Think of everything between the previous layer and the next layer as a black box.



Ktunotes.in

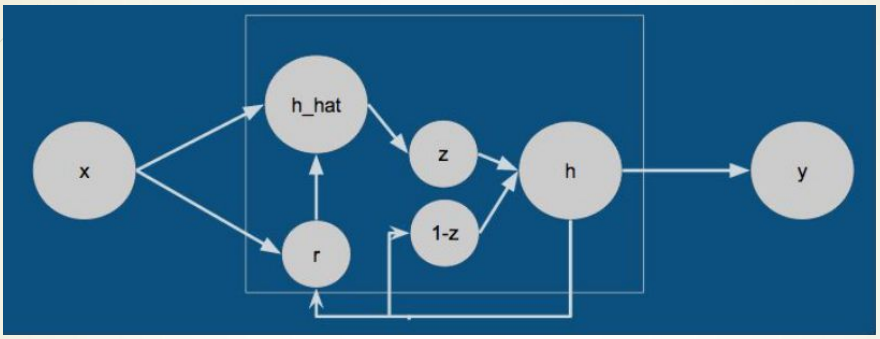
- In the **rated recurrent network**, we add a rating operation between what would've been the output of the simple recurrent network, and the previous output value.



- This **new operation as a gate**. Since it has to take on a value between 0 and 1, and the other gate has to take on 1 minus that value, it's a gate that is choosing between 2 things - taking on the old value, or taking on the new value.
- The result is we get a mixture of both.

Ktunotes.in

- the gated recurrent unit or GRU.-The architecture simply requires that we add one more gate



--Weight matrices multiplied by inputs and passed through nonlinear functions and gates.

Ktunotes.in

$$r_t = \sigma(x_t W_{xr} + h_{t-1} W_{hr} + b_r)$$
$$z_t = \sigma(x_t W_{xz} + h_{t-1} W_{hz} + b_z)$$
$$\hat{h}_t = g(x_t W_{xh} + (r_t \odot h_{t-1}) W_{hh} + b_h)$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \hat{h}_t.$$

- the **circle with the dot** in it is another way of representing **element-wise multiplication.**
- the "**update gate" z(t)**--balances how much of the previous hidden value and how much of the new candidate hidden value combines to get the new hidden value
- r(t), or the "reset gate"--The "reset gate" is multiplied by the previous hidden state value - it controls how much of the previous hidden state we will consider when we create the new candidate hidden value

Ktunotes.in

- it has the ability to “reset” the hidden value
  - If r(t) = 0, then we get h_hat(t) = f(x(t) Wxh + bh), which would be as if x(t) were the beginning of a new sequence.
  - since h(t) will be a combination of h_hat(t) and h(t-1), controlled by the “update gate” z(t).

Ktunotes.in

- The main difference with the LSTM is that a single gating unit simultaneously controls the forgetting factor and the decision to update the state unit
- The update equations are the following:

$$h_i^{(t)} = u_i^{(t-1)} h_i^{(t-1)} + (1 - u_i^{(t-1)}) \sigma \left(b_i + \sum_j U_{i,j} x_j^{(t-1)} + \sum_j W_{i,j} r_j^{(t-1)} h_j^{(t-1)} \right), \quad (10.45)$$

- where *u* stands for "update" gate and *r* for "reset" gate. Their value is defined as usual:

$$u_i^{(t)} = \sigma \left(b_i^u + \sum_j U_{i,j}^u x_j^{(t)} + \sum_j W_{i,j}^u h_j^{(t)} \right) \quad (10.46)$$

$$r_i^{(t)} = \sigma \left(b_i^r + \sum_j U_{i,j}^r x_j^{(t)} + \sum_j W_{i,j}^r h_j^{(t)} \right). \quad (10.47)$$

Ktunotes.in

- The **reset and updates gates can individually "ignore" parts of the state vector**
- The update gates can linearly gate any dimension, thus **choosing to copy it** (at one extreme of the sigmoid) or **completely ignore it** (at the other extreme) by replacing it by the **new "target state" value** (towards which the leaky integrator wants to converge).
- The **reset gates control which parts of the state get used to compute the next target state**, introducing an additional nonlinear effect in the relationship between past state and future state.

Ktnotes.in

- The **reset gate (or forget gate)** output could be shared across multiple hidden units
- The product of a **global gate (**covering a whole group of units, such as an entire layer) and a **local gate** (per unit) could be used to combine global control and local control.

Ktunotes.in